

Resumo do algoritmo

O algoritmo percorre o vetor `v` de `0` até `n-1` com passo `p`, acumulando a soma parcial `soma`. Em seguida, retorna `sqrt(soma)` somado ao resultado de uma chamada recursiva com novo tamanho `n' = soma % n`.

Assim, cada nível de recursão tem custo dominante linear em relação a `n/p` devido ao laço, e o próximo subproblema é definido pelo resto da divisão, isto é, um valor no intervalo `[0, n-1]`.

Importa bibliotecas e define uma função auxiliar para calcular a soma feita pelo laço `for (i=0; i<n; i=i+p)` do algoritmo.

```
In [2]: import math
from typing import List

def soma_com_passo(v: List[int], n: int, p: int) -> int:
    """Replica a soma do laço do enunciado: i = 0, p, 2p, ... < n."""
    soma = 0
    for i in range(0, n, p):
        soma += v[i]
    return soma
```

Implementa diretamente a função recursiva `foo` do enunciado.

```
In [ ]: def foo(v: List[int], n: int, p: int) -> float:
    if n <= 0:
        return 0.0

    soma = soma_com_passo(v, n, p)
    return math.sqrt(soma) + foo(v, soma % n, p)
```

Executa um exemplo simples para mostrar o comportamento da função implementada.

```
In [6]: v_exemplo = [3, 1, 4, 1, 5, 9, 2, 6, 5]
n_exemplo = len(v_exemplo)
p_exemplo = 2

print("soma_com_passo:", soma_com_passo(v_exemplo, n_exemplo, p_exemplo))
print("foo:", foo(v_exemplo, n_exemplo, p_exemplo))

soma_com_passo: 19
foo: 6.090949751109552
```

Resultados pedidos no enunciado

1) Relação de recorrência

Para `n > 0`, o laco executa aproximadamente `n/p` iterações. Assim, o custo por chamada e:

$$\Theta\left(\frac{n}{p}\right)$$

Como a proxima chamada usa `n' = soma % n`, com `0 \le n' < n`, a recorrência pode ser escrita como:

$$T(n) = T(n') + \Theta\left(\frac{n}{p}\right), \quad 0 \leq n' < n$$

com caso base:

$$T(0) = \Theta(1)$$

2) Complexidade de tempo (melhor e pior caso)

- Melhor caso: quando `n' = 0` ja na primeira chamada (isto e, `soma % n = 0`).

$$T(n) = \Theta\left(\frac{n}{p}\right)$$

- Pior caso: quando a redução de `n` e minima a cada passo (por exemplo, `n' = n-1`, depois `n-2`, ...).

$$T(n) = \sum_{k=1}^n \Theta\left(\frac{k}{p}\right) = \Theta\left(\frac{n^2}{p}\right)$$

Se p for constante, isso equivale a $\Theta(n^2)$.

3) Complexidade de espaço (melhor e pior caso)

- Espaço extra por chamada (variáveis locais): $\Theta(1)$.
- Espaço de pilha depende da profundidade da recursão:
 - Melhor caso: profundidade constante (1 chamada recursiva efetiva): $\Theta(1)$.
 - Pior caso: profundidade linear em `n`: $\Theta(n)$.

4) Questão sobre resto de divisão (`x % 7`)

Para divisor fixo positivo `m`, os possíveis resultados de `x % m` sao todos os inteiros no intervalo:

$$\{0, 1, 2, \dots, m - 1\}$$

Logo, para `x % 7`, os possíveis resultados sao:

$$\{0, 1, 2, 3, 4, 5, 6\}$$